Tutorial 3: Java, JDBC & Oracle

CSCI3170 Introduction to Database Systems

Learning Objectives

By the end of this tutorial, you should be able to:

1. Use Terminal to transfer lab files, log in to the school Linux host, and connect SQL*Plus to the Oracle server.
2. Create tables, load data, SELECT, INSERT, UPDATE, and DELETE **via command line.**
3. Create tables, load data, SELECT, INSERT, UPDATE, and DELETE **via Java**.
4. Use `Connection`, `Statement`, and `PreparedStatement`, including typed parameters and statement reuse.
5. Compare both clients' results and build two Java programs that process data and commit updates to Oracle.

The demonstration sequence

1. **Command-line round:** create six tables, import the dataset, save a schema/data snapshot, and demonstrate CRUD in SQL*Plus(Command-line database client provided by Oracle).
2. **Reset in Terminal:** drop only these six lab tables and verify that none remains. Preserve the output files from the first round.
3. **Java round:** connect with JDBC, recreate all six tables, import the same dataset, demonstrate prepared queries, and repeat CRUD.
4. **Compare:** check table definitions, keys, all imported rows, and each intermediate CRUD result.
5. **Process data:** run the budget and maintenance-workload cases and verify committed changes from SQL*Plus.

File	Role
`data.zip` / `data/`	Original schema and data: eight SQL files.
`OracleJDBCExample.java`	Reference
`tut3JDBC.java`	Runnable setup, CRUD, prepared query, and processing cases.
`tut3_snapshot.sql`	Save schema, keys, all rows, and counts for comparison.
`tut3_crud.sql`	Execute and capture the SQL*Plus CRUD round.
`tut3_reset.sql`	Drop the six lab tables, child tables first.

1. Connect from Terminal and Uploading Tutorial Files to CSE Host

1a. Open Terminal on your Device. Press Command–Space, type Terminal, and press Return. A prompt ending in `%` or `$` is a shell prompt; do not type the prompt itself. In the commands below, replace `YOUR_CSE_ID` and `YOUR_ORACLE_ID` with your own IDs.

1b. Transfer the files from your device to the school host. In macOS/Linux Terminal, use your local lab directory (change the example path if needed):

```
# Navigating to local source folder where tutorial files are.
cd "<local directory>"
pwd
ls
# create a destination folder on CSE server to receive files.
# Please connect to CSE VPN first of you want to access CSE Server.
ssh YOUR_CSE_ID@linux1.cse.cuhk.edu.hk "mkdir -p ~/csci3170-tut3"
# transfer all files.
scp data.zip tut3JDBC.java OracleJDBCExample.java tut3_*.sql \
    YOUR_CSE_ID@linux1.cse.cuhk.edu.hk:~/csci3170-tut3/
```

Windows Command Prompt (CMD)

Press Win–R, type `cmd`, and press Enter. With Windows OpenSSH installed, run the following commands. Replace the example local path and `YOUR_CSE_ID`; connect to the CSE VPN first if required.

```
# Change to the local tutorial folder, including its drive.
cd /d "C:\path\to\tut3"
# Show the current folder and list its files.
dir
# Create the destination folder on the CSE server.
ssh YOUR_CSE_ID@linux1.cse.cuhk.edu.hk "mkdir -p ~/csci3170-tut3"
# Upload the tutorial files (enter this as one command).
scp data.zip tut3JDBC.java OracleJDBCExample.java tut3_*.sql YOUR_CSE_ID@linux1.cse.cuhk.edu.
    hk:~/csci3170-tut3/
# Log in to the remote Linux shell.
ssh YOUR_CSE_ID@linux1.cse.cuhk.edu.hk
```

In CMD, `cd /d` changes folder and drive, `cd` alone replaces `pwd`, and `dir` replaces `ls`. The long `scp` command may wrap visually above; enter it on one line. For later local paths, replace `$HOME` with `%USERPROFILE%`. After the final `ssh`, continue with Section 1c from `cd ~/csci3170-tut3`; you are now in Linux, so use its commands unchanged.

Please use your **CSE password** for SSH/SCP. Password typing normally has no visible characters. Use the school network or the course-approved VPN if off-campus access requires it.

1c. Access remote CSE Linux shell. Confirm the data and initialize Oracle: As noted previously, you can log in using just `sqlplus YOUR_ORACLE_ID@db18.cse.cuhk.edu.hk`, then enter your Oracle password when prompted.

```
ssh YOUR_CSE_ID@linux1.cse.cuhk.edu.hk
# navigating to destination folder and checking data.zip.
cd ~/csci3170-tut3
unzip -n data.zip
ls data
# connecting to oracle db server.
tcsh
source /opt1/oracle122/setup
sqlplus YOUR_ORACLE_ID@db18.cse.cuhk.edu.hk
# Enter your Oracle password when prompted.
```

2. Understand the schema and loader scripts

All eight original files are reproduced in full in the appendix. Here we explain them before execution.

`create_table.sql`: remove old tables, define six tables

The file first drops RESEARCH and MAINTAIN, then APPLICATION, PROJECT, ENGINEER, and OFFICE. It then creates the six tables; for example:

```
CREATE TABLE OFFICE (
    OFFICEID INTEGER NOT NULL PRIMARY KEY,
    ONAME VARCHAR(50) NOT NULL,
    OADDRESS VARCHAR(30) NOT NULL,
    OPHONENUM VARCHAR(10) NOT NULL
);
CREATE TABLE MAINTAIN (
    APPID INTEGER NOT NULL REFERENCES APPLICATION(APPID),
    ENGINEERID INTEGER NOT NULL REFERENCES ENGINEER(ENGINEERID)
);
```

Table	Meaning and constraints in the supplied file
OFFICE	Office ID (primary key), name, address, phone number.
ENGINEER	Engineer ID (primary key), name, integer salary, office ID.
PROJECT	Project ID (primary key), office ID, name, category, integer budget and expenditure.
APPLICATION	App ID (primary key), name, integer price, integer date in YYYYMMDD form, project ID.
MAINTAIN	App–engineer pairs. Both columns are foreign keys.
RESEARCH	Project–engineer pairs. Both columns are foreign keys.

`add.sql`: invoke the six data files

```
@addapplication
@addengineer
@addmaintain
@addoffice
@addproject
@addresearch
```

These are SQL*Plus commands: `@name` runs `name.sql`. They are not SQL statements and cannot be sent directly to JDBC. Run `@add.sql` with the shell working directory set to `data/`; its nested paths are relative to that directory.

3. Understand the Data File(s) inside data.zip

Each statement uses `INSERT INTO table VALUES (...)` without a column list, so values must follow the column order in `create_table.sql`. The examples below are exact excerpts; the appendix contains every original row.

`addoffice.sql`: 8 offices

```
INSERT INTO OFFICE VALUES(7,'Database Group','P.O. Box 610, 3867 Lobortis Road','27159250');
```

Order: OFFICEID, ONAME, OADDRESS, OPHONENUM. This row is office 7, Database Group. The phone is a string, not a number used in arithmetic.

`addengineer.sql`: 89 engineers

```
INSERT INTO ENGINEER VALUES(10,'Chan Ling Yun',28350,1);
```

Order: ENGINEERID, ENAME, ESALARY, OFFICEID. Engineer 10 starts with salary 28350 and belongs to office 1. The workload case will update this salary.

`addproject.sql`: 14 projects

```
INSERT INTO PROJECT VALUES(12,1,'CoffeeScript','Web',19925,29325);
```

Order: PROJECTID, OFFICEID, PNAME, PCATEGORY, PBUDGET, PEXPENDITURE. Project 12 has spent 29325 against a budget of 19925; it is a candidate for the budget case.

`addapplication.sql`: 23 applications

```
INSERT INTO APPLICATION VALUES(19,'Barbara',42196,20131027,12);
```

Order: APPID, ANAME, APRICE, ADATE, PROJECTID. App 19 belongs to project 12; 20131027 represents 27 October 2013. The supplied schema stores ADATE as INTEGER, not Oracle DATE.

`addmaintain.sql`: 26 app–engineer pairs

```
INSERT INTO MAINTAIN VALUES(19,7);
INSERT INTO MAINTAIN VALUES(19,93);
```

Order: APPID, ENGINEERID. Engineers 7 and 93 both maintain app 19. Referenced app/engineer rows must already exist. The Java workload case groups these pairs by engineer.

`addresearch.sql`: 71 project–engineer pairs

```
INSERT INTO RESEARCH VALUES(11,110);
INSERT INTO RESEARCH VALUES(11,130);
```

Order: PROJECTID, ENGINEERID. Engineers 110 and 130 both research project 11. Referenced project/engineer rows must already exist. In total, the six data files insert **231 rows**.

4. [DB operations via. Terminal DEMO]: create tables/ import data

Start from the lab directory on the remote host. Close any earlier SQL*Plus session with `EXIT`. The supplied reset script removes only the six lab tables.

```
cd ~/csci3170-tut3
sqlplus YOUR_ORACLE_ID@db18.cse.cuhk.edu.hk
```

```
@tut3_reset.sql
-- Expected: LAB_TABLES_REMAINING = 0.
EXIT
```

Create and load from the data directory. Run in the remote shell:

```
cd data
sqlplus YOUR_ORACLE_ID@db18.cse.cuhk.edu.hk
```

What does `SPOOL` do? This SQL*Plus command saves displayed output, including query results and messages, to a text file until `SPOOL OFF`. Here, `../sqlplus_load.log` is in the parent lab directory on the remote CSE host, and `REPLACE` overwrites an existing log. It records output for later checking; it does not commit database changes.

```
SPOOL ../sqlplus_load.log REPLACE
@create_table.sql
-- The six initial DROP statements report ORA-00942 because we reset.
-- All six CREATE TABLE statements must succeed. Stop if they do not.
-- From here, a SQL/PLSQL error triggers rollback and exits SQL*Plus.
-- SQL.SQLCODE supplies the error status; only uncommitted changes undo.
-- This rule does not check earlier commands or SQL*Plus command errors.
WHENEVER SQLERROR EXIT SQL.SQLCODE ROLLBACK
ALTER TABLE OFFICE MODIFY (OADDRESS VARCHAR(50));
@add.sql
-- COMMIT saves all pending changes in this session's transaction.
-- Other sessions can then read the imported data in new queries.
-- A later ROLLBACK cannot undo these committed inserts.
COMMIT;
SELECT 'OFFICE' AS TAB, COUNT(*) AS N FROM OFFICE
UNION ALL SELECT 'ENGINEER', COUNT(*) FROM ENGINEER
UNION ALL SELECT 'PROJECT', COUNT(*) FROM PROJECT
UNION ALL SELECT 'APPLICATION', COUNT(*) FROM APPLICATION
UNION ALL SELECT 'MAINTAIN', COUNT(*) FROM MAINTAIN
UNION ALL SELECT 'RESEARCH', COUNT(*) FROM RESEARCH;
SPOOL OFF
EXIT
```

Schema repair in both rounds: office 7 has a 32-character address, exceeding the original `VARCHAR(30)`. The ALTER above expands it to 50 before import; Our Java example later also performs the same ALTER operations, leaving the original files remain unchanged.

Expected counts: OFFICE 8; ENGINEER 89; PROJECT 14; APPLICATION 23; MAINTAIN 26; RESEARCH 71.

Save the baseline for the later Java comparison. Return to the lab directory in the remote shell and reconnect:

```
cd ..
sqlplus YOUR_ORACLE_ID@db18.cse.cuhk.edu.hk
```

```
@tut3_snapshot.sql sqlplus_setup.txt
```

This helper writes 23 column descriptions, 8 primary/foreign-key column records, 231 data rows, and 6 row counts to `sqlplus_setup.txt`. Keep this SQL*Plus session open, we continue with a basic

CRUD walkthrough on the next page.

4.1. [DB operations via. Terminal DEMO]: CRUD in SQL*Plus

Continue in the SQL*Plus session from the previous page. Start with the freshly loaded and committed data; OFFICEID 99 must not exist. No other client should modify OFFICE during this comparison.

Live walkthrough. First read the baseline, then read again after each change. Select all columns so a difference in any stored value is visible.

```
-- Keep the demo changes pending until COMMIT or ROLLBACK.
SET AUTOCOMMIT OFF
-- Read: 8 original rows, ordered by primary key.
SELECT OFFICEID, ONAME, OADDRESS, OPHONENUM
FROM OFFICE ORDER BY OFFICEID;

-- Create: insert one new office.
INSERT INTO OFFICE (OFFICEID, ONAME, OADDRESS, OPHONENUM)
VALUES (99, 'JDBC Demo Lab', 'Room 3170', '20000000');
SELECT * FROM OFFICE WHERE OFFICEID = 99;
SELECT COUNT(*) FROM OFFICE;

-- Update: change that office's name and phone number.
UPDATE OFFICE SET ONAME = 'JDBC Demo Lab Updated',
                  OPHONENUM = '29999999'
WHERE OFFICEID = 99;
SELECT * FROM OFFICE WHERE OFFICEID = 99;
SELECT COUNT(*) FROM OFFICE;

-- Delete: remove the demo office.
DELETE FROM OFFICE WHERE OFFICEID = 99;
SELECT * FROM OFFICE WHERE OFFICEID = 99;
SELECT COUNT(*) FROM OFFICE;
-- ROLLBACK cancels all uncommitted changes in this transaction.
-- This demo leaves the previously committed seed data unchanged.
-- It cannot undo committed data or successful CREATE/ALTER/DROP.
ROLLBACK;
```

Expected observations (address stays `Room 3170`):

Stage	Total rows	Name for ID 99	Phone for ID 99
Baseline	8	No row	—
INSERT	9	JDBC Demo Lab	20000000
UPDATE	9	JDBC Demo Lab Updated	29999999
DELETE	8	No row	—

Each INSERT, UPDATE, and DELETE affects exactly one row.

Capture a reproducible trace. After the walkthrough, run the supplied script from the lab directory:

```
@tut3_crud.sql
EXIT
```

The script repeats these operations, records *all OFFICE rows* in primary-key order at each stage, writes `sqlplus_crud.txt`, and rolls back. Its output format is `ROW|id|name|address|phone`, with `STAGE` and `COUNT` lines. Both trace files are replaced on each run.

4.2. [DB operations via. Terminal DEMO]: Cleanup

DELETE removes rows but leaves table definitions; ROLLBACK cannot undo committed data or CREATE TABLE. So we need to reset before the Java round. **4.2a. Preserve the first round's files.** At the remote shell:

```
cd ~/csci3170-tut3
ls -l sqlplus_load.log sqlplus_setup.txt sqlplus_crud.txt
cat sqlplus_crud.txt
sqlplus YOUR_ORACLE_ID@db18.cse.cuhk.edu.hk
```

4.2b. Reset through SQL*Plus in the same terminal window:

```
@tut3_reset.sql
SELECT COUNT(*) AS LAB_TABLES_REMAINING FROM USER_TABLES
WHERE TABLE_NAME IN
 ('OFFICE','ENGINEER','PROJECT','APPLICATION','MAINTAIN','RESEARCH');
-- Required result: 0.
EXIT
```

What the reset script does. It first rolls back any pending DML, then performs these drops in dependency order:

```
DROP TABLE RESEARCH;
DROP TABLE MAINTAIN;
DROP TABLE APPLICATION;
DROP TABLE PROJECT;
DROP TABLE ENGINEER;
DROP TABLE OFFICE;
```

The supplied `tut3_reset.sql` wraps each DROP in a small PL/SQL exception handler: it tolerates only ORA-00942 (table already absent), prints what happened, and rethrows every other error. It checks the final table count. It does not use CASCADE to remove unrelated dependencies. Read it from the remote shell with `cat tut3_reset.sql`.

Checkpoint before Java

- All six original lab tables are absent: table count is 0.
- The eight SQL source files remain in `data/`.
- The first round's setup and CRUD traces remain on disk.
- Do not run `create_table.sql` again here. Java will now create the tables.

DDL implicitly commits. If reset or setup stops partway through, earlier successful DROP/CREATE statements remain effective. Resolve the reported problem, rerun the reset, and start Java setup again. Do not expect a Java rollback to remove successfully created tables.

5. [DB operations via. Java DEMO]: Connect, compile and connect

Driver and JDK. Use a course-approved Oracle JDBC JAR. If the JAR is on your local device, transfer it from a *second local Terminal window*:

```
cd "<local directory>"
scp ojdbc11.jar YOUR_CSE_ID@linux1.cse.cuhk.edu.hk:~/csci3170-tut3/
```

Verify in the **remote shell**:

```
cd ~/csci3170-tut3
java -version
javac -version
ls -l tut3JDBC.java ojdbc11.jar data/create_table.sql
javac -cp .:ojdbc11.jar tut3JDBC.java
java -cp .:ojdbc11.jar tut3JDBC setup
```

The classpath uses `:` on Linux/macOS. Run in a real terminal without redirecting input/output: the program prompts for an Oracle username and hidden password, and writes its own trace files. Use the **same Oracle account and service** as the SQL*Plus round.

For a guided tour of the key methods and transaction flow, see the Java source reading guide in the appendix.

What is a Connection?

A `Connection` is the database session. The URL selects the server/service; credentials select your account. The code extends the supplied `OracleJDBCExample.java`:

```
String url = "jdbc:oracle:thin:@//db18.cse.cuhk.edu.hk:1521/oradb.cse.cuhk.edu.hk";
Class.forName("oracle.jdbc.OracleDriver");
try (Connection c = DriverManager.getConnection(url, username, password)) {
    c.setAutoCommit(false);
    // Create tables, load data, or run statements through c.
}
```

`username` and `password` are read from the terminal, not stored in the source. Try-with-resources closes the connection. The program disables JDBC's default auto-commit for DML, commits successful seed imports and processing cases, and rolls back failed DML.

Connection versus Prepared Statement

There is no standard JDBC type called "PreparedConnection". The pattern is **open a Connection, then call its `prepareStatement(sql)` method**. A `PreparedStatement` belongs to that connection and binds values into a fixed SQL template. It is not a second login or a second connection.

SQL versus client commands. Java sends SQL to the same Oracle server as SQL*Plus. It can execute CREATE TABLE and CRUD with the same permissions. SQL*Plus commands such as `SPOOL`, `SET`, and `@add.sql` are client commands; do not pass them to JDBC. JDBC SQL strings have no trailing semicolon.

5.1. [DB operations via. Java DEMO]: (re)create tables and (re)load all data

`tut3JDBC setup` refuses to start if any of the six lab tables exists. Its `initialize()` method reads `data/create_table.sql`, skips the DROP statements already performed in Terminal, and executes each of the six CREATE TABLE statements using JDBC. The equivalent OFFICE call is:

```
try (Statement s = c.createStatement()) {
    s.executeUpdate("CREATE TABLE OFFICE ("
        + "OFFICEID INTEGER NOT NULL PRIMARY KEY, "
        + "ONAME VARCHAR(50) NOT NULL, "
        + "OADDRESS VARCHAR(30) NOT NULL, "
        + "OPHONENUM VARCHAR(10) NOT NULL)");
}
```

For DDL, `executeUpdate()` returns 0; that does not indicate a failure. This example is part of setup; do not execute it again after OFFICE exists.

Actual loop in the runnable program:

```
try (Statement s = c.createStatement()) {
    for (String sql : statements("create_table.sql")) {
        if (sql.startsWith("DROP TABLE ")) continue;
        if (!sql.startsWith("CREATE TABLE "))
            throw new SQLException("Unexpected DDL");
        s.executeUpdate(sql);
    }
}
```

Next, the java program makes the same adjustment to the schema before data import:

```
s.executeUpdate("ALTER TABLE OFFICE MODIFY (OADDRESS VARCHAR(50))");
```

The `statements()` helper reads a UTF-8 file in `data/`, splits its statements on semicolons, and removes the terminators. It's usage is limited, but will work with our supplied simple scripts, which have no semicolons inside strings; it is not a general SQL*Plus or PL/SQL parser.

Import data through Java. The program executes every INSERT from the six seed files, in this table order: OFFICE, ENGINEER, PROJECT, APPLICATION, MAINTAIN, RESEARCH. This parent-first order respects the foreign keys and produces the same data as `add.sql`. The fixed, trusted seed SQL uses `Statement`; the query and CRUD demonstrations below use bound parameters.

```
// Inside the loop over the six seed files:
for (String sql : statements(files[i])) {
    requireOne(s.executeUpdate(sql)); // Each INSERT must affect one row.
}
// main() commits the seed data, then saves jdbc_setup.txt.
```

Expected: 6 tables created; per-file insert counts 8, 89, 14, 23, 26, 71; 231 total rows committed. The program then writes `jdbc_setup.txt`.

Compare both creation/import rounds from the remote shell:

```
diff -u -b -B sqlplus_setup.txt jdbc_setup.txt
```

Expected: no differences, exit status 0. Both files must contain 23 `COLUMN`, 8 `KEY`, 231 `DATA`, and 6 `COUNT` lines. Generated constraint names are excluded; primary/foreign-key columns and reference targets are included.

5.2. [DB operations via. Java DEMO]: PreparedStatement: bind and reuse parameters

Run the read-only demonstration after setup:

```
java -cp .:ojdbc11.jar tut3JDBC prepared
```

The same prepared statement is executed twice, first with office ID 1 and then with ID 7:

```
try (PreparedStatement p = c.prepareStatement(
        "SELECT OFFICEID, ONAME FROM OFFICE WHERE OFFICEID = ?")) {
    for (int id : new int[] {1, 7}) {
        p.setInt(1, id);
        try (ResultSet r = p.executeQuery()) {
            while (r.next()) {
                System.out.println(r.getInt(1) + " | " + r.getString(2));
            }
        }
    }
}
```

Expected output:

```
1 | Cloud Computing Research Center
7 | Database Group
```

1. `c.prepareStatement(...)` constructs the parameterized SQL statement on the existing connection.
2. The unquoted `?` is a value placeholder. `setInt(1, id)` binds the first placeholder; numbering starts at 1. Use `setString` for names and other text.
3. `executeQuery()` returns a `ResultSet`; `next()` advances one row. The SQL string is not passed again to `executeQuery()`.
4. After closing the result set, change the parameter and reuse `p`. Close the prepared statement when finished.

Operation	JDBC method and result
SELECT	`executeQuery()` returns rows in a `ResultSet`.
INSERT/UPDATE/DELETE	`executeUpdate()` returns the affected row count.
CREATE TABLE	`Statement.executeUpdate(sql)` executes DDL.
Save/undo DML	`Connection.commit()` / `rollback()`.

Bind data values instead of concatenating input into SQL. For example, a name containing an apostrophe is passed with `setString`; it remains data rather than SQL syntax. Parameters cannot replace a table or column name, and `'?'` is a literal question mark rather than a bind marker. Prepared statements can be reused, but do not assume they are always faster for every query.

Next: the CRUD method uses three more prepared statements: four parameters for INSERT, three for UPDATE, and one for DELETE. The original row-reading logic is retained in `snapshot()`, which selects every OFFICE column in OFFICEID order.

5.3. [DB operations via. Java DEMO]: repeat with Java and compare

After Java setup and the prepared-query demo, run the following code.

```
java -cp .:ojdbc11.jar tut3JDBC crud
```

Its `crud()` method executes the following statements;

```
snapshot(c, trace, "BASELINE"); //see note1 below.
try (PreparedStatement p = c.prepareStatement(
        "INSERT INTO OFFICE (OFFICEID, ONAME, OADDRESS, OPHONENUM) " +
        "VALUES (?, ?, ?, ?)")) {
    p.setInt(1, 99);
    p.setString(2, "JDBC Demo Lab");
    p.setString(3, "Room 3170");
    p.setString(4, "20000000");
    requireOne(p.executeUpdate()); //see note2 below.
}
snapshot(c, trace, "INSERT");
try (PreparedStatement p = c.prepareStatement(
        "UPDATE OFFICE SET ONAME = ?, OPHONENUM = ? WHERE OFFICEID = ?")) {
    p.setString(1, "JDBC Demo Lab Updated");
    p.setString(2, "29999999");
    p.setInt(3, 99);
    requireOne(p.executeUpdate());
}
snapshot(c, trace, "UPDATE");
try (PreparedStatement p = c.prepareStatement(
        "DELETE FROM OFFICE WHERE OFFICEID = ?")) {
    p.setInt(1, 99);
    requireOne(p.executeUpdate());
}
snapshot(c, trace, "DELETE");
// main() rolls back the CRUD demo in Section 5.3 after writing the trace.
```

Notes:

1. `snapshot()` performs the ordered SELECT after each operation and writes the same trace format as SQL*Plus.
2. `requireOne(n)` throws an exception unless `n == 1`. The complete program closes statements and result sets with try-with-resources, and rolls back on an exception.

6. Check if both approaches (Java and Terminal) were equivalent

From the shell, compare the generated files:

```
diff -u -b -B sqlplus_crud.txt jdbc_crud.txt
```

Expectation: no differences (exit status 0). The flags ignore whitespace layout and blank lines introduced by SQL*Plus. Check that both files contain all four stages and row counts `8, 9, 9, 8`; an empty or incomplete trace is not a successful experiment. The fresh dataset produces 34 `ROW` lines in each trace.

If there is a difference, identify the first differing stage. Check the account/service, initial data, values, WHERE condition, and ordering. A duplicate-key error usually means ID 99 was already present or an earlier transaction was committed.

Explain the result

Why might checking only the final eight rows insufficient? An incorrect INSERT or UPDATE can be hidden by the later DELETE. Matching every intermediate snapshot demonstrates that the same database operations produce the same data through both clients.

If time is limited, leave the following examples for self-study.

Java Application Example One: a project budget rescue tool

Scenario. A project manager wants a program that flags overspending and proposes a revised budget. Use PROJECT from the supplied dataset.

Rule. If expenditure exceeds budget, set the new budget to expenditure plus 10%, rounded *up* to a multiple of 100. Otherwise keep the existing budget. This is a hypothetical policy we made up for this demonstration.

Try it:

```
java -cp .:ojdbc11.jar tut3JDBC budget
```

Expected changes from fresh data:

ID	Project	Expenditure	Old budget	New budget
9	Flume	27500	19838	30300
12	CoffeeScript	29325	19925	32300
13	ApacheDrill	28034	20115	30900
15	FalseCrypt	27515	20044	30300
20	GFS	27442	19888	30200
26	EC3	29427	20005	32400
27	Rapyuta	24622	19886	27100
29	MS-IIS	29424	20121	32400

The program prints the proposed changes, updates **8 projects**, and commits. The other 6 budgets stay unchanged. With unchanged expenditures, running this case again updates 0 projects because none is still over budget.

Explanation:

Read → process → write. Fetch `PROJECTID, PNAME, PBUDGET, PEXPENDITURE`; calculate in Java; collect changed rows; close the result set; then execute parameterized updates.

```
static int newBudget(int oldBudget, int spent) {
    if (spent <= oldBudget) return oldBudget;
    return Math.toIntExact(((spent * 110L + 9999) / 10000) * 100);
}
// Inside the ResultSet loop:
int next = newBudget(old, spent);
if (next != old) changes.add(new int[] {id, old, next});

// After the ResultSet has closed:
try (PreparedStatement p = c.prepareStatement(
        "UPDATE PROJECT SET PBUDGET = ? " +
        "WHERE PROJECTID = ? AND PBUDGET = ?")) {
    for (int[] row : changes) {
        p.setInt(1, row[2]);   // computed budget
        p.setInt(2, row[0]);   // project ID
        p.setInt(3, row[1]);   // budget that Java read
        requireOne(p.executeUpdate());
    }
}
```

Notes:

1. The `long` intermediate avoids integer multiplication overflow for the supplied values. For example, $29325 \times 1.10 = 32257.5$, which rounds up to 32300.
2. The old-value condition detects a changed budget; an affected-row count of zero aborts and rolls back the run.
3. We use a simple single writer for this lab: this condition does not protect other input columns from concurrent edits.

Java Application Example Two: a maintenance workload reward tool

Scenario. Reward engineers maintaining several applications. Read MAINTAIN into a Java map from engineer ID to a set of application IDs, then read ENGINEER to calculate new salaries.

Rule. Engineers maintaining at least 3 distinct apps receive an increase of 500 per app, capped at 5 apps (maximum increase: 2500). Engineers below that threshold receive no change. This is a one-time lab salary adjustment.

Try it:

```
java -cp .:ojdbc11.jar tut3JDBC workload
```

Expected changes from fresh data:

ID	Engineer	Apps	Old salary	Increase	New salary
10	Chan Ling Yun	6	28350	2500	30850
17	Ye Huanbang	4	35850	2000	37850
122	Jane Xiu	6	46100	2500	48600

The program updates **3 engineers** and commits; the other 86 salaries stay unchanged.

Explanation:

```
Map<Integer, Set<Integer>> appsByEngineer = new HashMap<>();
// r comes from: SELECT ENGINEERID, APPID FROM MAINTAIN
while (r.next()) {
    int id = r.getInt("ENGINEERID");
    appsByEngineer.computeIfAbsent(id, k -> new HashSet<>())
                  .add(r.getInt("APPID"));
}
```

A `Set` counts each application once. This matters because the supplied MAINTAIN table has no primary key to reject duplicate engineer/application pairs. Engineers missing from the map have zero apps.

```
static int newSalary(int oldSalary, int apps) {
    if (apps < 3) return oldSalary;
    return Math.addExact(oldSalary, 500 * Math.min(apps, 5));
}
// Inside the ENGINEER ResultSet loop:
Set<Integer> apps = appsByEngineer.get(id);
int count = apps == null ? 0 : apps.size();
int next = newSalary(old, count);
if (next != old) changes.add(new int[] {id, old, next});

// After all reads are complete:
try (PreparedStatement p = c.prepareStatement(
        "UPDATE ENGINEER SET ESALARY = ? " +
        "WHERE ENGINEERID = ? AND ESALARY = ?")) {
    for (int[] row : changes) {
        p.setInt(1, row[2]);
        p.setInt(2, row[0]);
        p.setInt(3, row[1]);
        requireOne(p.executeUpdate());
    }
}
```

Notes:

If you ran the above program again: the rule would certainly apply the increase applied again; which in this case is problematic. A production safeguard extension could record a unique reward period to prevent duplicate awards. In our case, reload the lab dataset before repeating the original demonstration.

Checking the above saved changes from Java, via SQL*Plus

After each Java case prints `COMMITTED`, reconnect to SQL*Plus using the same account. You may query your Oracle Database using via command line to verify that Oracle saved the values.

```
-- Case A: compare all 14 rows with the original data.
SELECT PROJECTID, PNAME, PBUDGET, PEXPENDITURE
FROM PROJECT ORDER BY PROJECTID;
--
SELECT COUNT(*) AS STILL_OVER_BUDGET
FROM PROJECT WHERE PEXPENDITURE > PBUDGET;
-- Expected: 0; the eight revised budgets match the Case A table.

-- Case B: independently check workload and saved salaries in SQL.
SELECT e.ENGINEERID, e.ENAME, e.ESALARY,
       COUNT(DISTINCT m.APPID) AS APPS
FROM ENGINEER e LEFT JOIN MAINTAIN m
  ON e.ENGINEERID = m.ENGINEERID
GROUP BY e.ENGINEERID, e.ENAME, e.ESALARY
HAVING COUNT(DISTINCT m.APPID) >= 3
ORDER BY e.ENGINEERID;
-- Expected: exactly the three rows in the Case B table.
SELECT SUM(ESALARY) AS TOTAL_SALARY FROM ENGINEER;
-- Expected: 2734050 (original 2727050 + 7000), after one run.
```

Check the exact budgets, the three target salaries, and the salary total. Compare other engineers with `addengineer.sql` to confirm they are unchanged.

Transaction behavior to explain during the demo

- JDBC connections initially use auto-commit. This program calls `setAutoCommit(false)` so a case's updates form one transaction.
- Sections 4 and 5.1 commit the seed data; the CRUD demos in Sections 4.1 and 5.3 roll back their changes. Section 4.2 drops the tables before Java rebuilds them. Java Application Examples One and Two each commit their completed updates.
- If any update fails or affects an unexpected number of rows, the program rolls back the case. It does not commit a partially completed set of updates.
- Changing a Java variable or collection does not change Oracle. The program must execute UPDATE and then commit before another session sees the saved values.
- Oracle DDL such as CREATE/DROP implicitly commits. Keep schema setup separate from these DML rollback demonstrations.

Checkpoints and discussion

1. Show both complete CRUD traces and the diff result. Explain why the queries use ORDER BY and why both runs need the same initial data.
2. For each case, identify the SQL read, Java computation, SQL write, and commit. Explain one unchanged record and one changed record.
3. Show the before/after values and the SQL*Plus verification. Explain why the budget case can be repeated but the salary case cannot be repeated safely as written.
4. Modify a policy (budget buffer, workload threshold, or cap), predict one result, run the Terminal reset and Java setup again, then verify it.

Troubleshooting: Missing driver: check the JAR/classpath. Login or connection failure: check Oracle credentials, school network access, and service details. Missing table: load data into the same account. Missing changes: check for `COMMITTED` and rerun the query.

References: Oracle's Prepared Statements and Transactions tutorials. See also Oracle's COMMIT reference.

Appendix: Java source reading guide

Reading `tut3JDBC.java`: follow the setup command

After running `java -cp .:ojdbc11.jar tut3JDBC setup`, open the source and start at `main(String[] args)`. Read the following parts in order; Section 5.1 examines the setup SQL in more detail. The command has already created and loaded the tables, so do not rerun setup without resetting them first.

1. The command selects a mode. `tut3JDBC` is the class containing `main`; `setup` becomes `args[0]`. The initial check accepts exactly one of five modes: `setup`, `prepared`, `crud`, `budget`, or `workload`. Only the selected branch runs, so setup does not also run the CRUD or processing demonstrations.

2. Read credentials and open one database session. `System.console()` reads the Oracle username and a password without displaying the password. The program uses `ORACLE_JDBC_URL` if set, otherwise `DEFAULT_URL`. `Class.forName(...)` loads the Oracle driver, and `DriverManager.getConnection(...)` opens the session represented by `Connection c`. The same `c` is passed to helper methods so their SQL runs in that session.

3. Follow the setup branch. These are the key calls, in execution order (file-opening and error checks are omitted here):

```
c.setAutoCommit(false);
initialize(c);
c.commit();
databaseSnapshot(c, trace);
```

`initialize(c)` first checks that none of the six lab tables exists. It then creates six tables, expands `OADDRESS` to 50, and imports the six data files with parent rows before dependent rows. `statements(file)` reads the supplied simple SQL files and removes semicolon terminators; Java executes the resulting SQL strings, not SQL*Plus commands such as `@add.sql`.

4. Distinguish executing SQL from reading its result. A `Statement` sends fixed SQL through `c`. `executeUpdate(sql)` is used for DDL and INSERT/UPDATE/DELETE; for each seed INSERT, `requireOne(...)` checks that exactly one row was affected. `executeQuery(sql)` returns a `ResultSet`; `r.next()` advances to a row, and getters such as `r.getInt(1)` read its columns. JDBC column indexes start at 1. Later methods use `PreparedStatement` to bind values to `?` placeholders.

5. Commit the data, then write the comparison file. `setAutoCommit(false)` prevents each INSERT from being committed separately. Once initialization succeeds, `c.commit()` saves the imported rows for later sessions. `databaseSnapshot(c, trace)` queries column definitions, keys, all data rows, and counts, then writes `jdbc_setup.txt` through `PrintWriter`. This file is a comparison trace, not a database commit. If writing it fails after `c.commit()`, the imported data is already committed.

6. Find cleanup and failure handling. The `catch` block attempts `c.rollback()` and rethrows the error. This undoes pending DML, but cannot undo committed rows or successful Oracle DDL; reset the lab tables before retrying a failed setup. The `try (...)` blocks automatically close connections, statements, result sets, and writers. In the other modes, `crud` ends with rollback, while successful `budget` and `workload` runs commit their updates.

Appendix: The original complete SQL files Used in this Tutorial

The original file contents are embedded here so the handout compiles on its own. Execute the actual files in `data/`; do not type page headers or line numbers. Sections 2–3 explain the purpose and fields of every file.

`create_table.sql`

```
DROP TABLE RESEARCH;
DROP TABLE MAINTAIN;
DROP TABLE APPLICATION;
DROP TABLE PROJECT;
DROP TABLE ENGINEER;
DROP TABLE OFFICE;

CREATE TABLE OFFICE (
        OFFICEID INTEGER NOT NULL PRIMARY KEY,
        ONAME VARCHAR(50) NOT NULL,
        OADDRESS VARCHAR(30) NOT NULL,
        OPHONENUM VARCHAR(10) NOT NULL
);

CREATE TABLE ENGINEER (
        ENGINEERID INTEGER NOT NULL PRIMARY KEY,
        ENAME VARCHAR(50) NOT NULL ,
        ESALARY INTEGER NOT NULL ,
        OFFICEID INTEGER NOT NULL
);

CREATE TABLE PROJECT (
        PROJECTID INTEGER NOT NULL PRIMARY KEY,
        OFFICEID INTEGER NOT NULL,
        PNAME VARCHAR(50) NOT NULL,
        PCATEGORY VARCHAR(30) NOT NULL,
        PBUDGET INTEGER NOT NULL,
        PEXPENDITURE INTEGER NOT NULL
);

CREATE TABLE APPLICATION (
        APPID INTEGER NOT NULL PRIMARY KEY,
        ANAME VARCHAR(50) NOT NULL,
        APRICE INTEGER NOT NULL,
        ADATE INTEGER NOT NULL,
        PROJECTID INTEGER NOT NULL
);

CREATE TABLE MAINTAIN (
        APPID INTEGER NOT NULL REFERENCES APPLICATION(APPID),
        ENGINEERID INTEGER NOT NULL REFERENCES ENGINEER(ENGINEERID)
);

CREATE TABLE RESEARCH (
        PROJECTID INTEGER NOT NULL REFERENCES PROJECT(PROJECTID),
        ENGINEERID INTEGER NOT NULL REFERENCES ENGINEER(ENGINEERID)
);
```

add.sql

```
@addapplication
@addengineer
@addmaintain
@addoffice
@addproject
@addresearch
```

addoffice.sql

```
INSERT INTO OFFICE VALUES(1,'Cloud Computing Research Center','7174 Aliquam, Ave','22983946');
INSERT INTO OFFICE VALUES(2,'Multimedia Research Center','970-3326 Cras Road','21041918');
INSERT INTO OFFICE VALUES(3,'Network Coding Research Center','143-2971 Eu, Av.','28051035');
INSERT INTO OFFICE VALUES(4,'Telecommunication Simulation Group','P.O. Box 839, 5711 Duis St.','27869901');
INSERT INTO OFFICE VALUES(5,'Advanced Computer Vision Institute','247-7897 Ligula. Avenue','22000062');
INSERT INTO OFFICE VALUES(6,'Big Data Group','P.O. Box 786, 8965 Et Ave','25473333');
INSERT INTO OFFICE VALUES(7,'Database Group','P.O. Box 610, 3867 Lobortis Road','27159250');
INSERT INTO OFFICE VALUES(8,'Compatibility Tools Group','214-807 Nulla Street','28964438');
```

addproject.sql

```
INSERT INTO PROJECT VALUES(11,5,'V8Javascript','Web',19903,15805);
INSERT INTO PROJECT VALUES(12,1,'CoffeeScript','Web',19925,29325);
INSERT INTO PROJECT VALUES(13,7,'ApacheDrill','Novel',20115,28034);
INSERT INTO PROJECT VALUES(15,5,'FalseCrypt','Security',20044,27515);
INSERT INTO PROJECT VALUES(1,6,'Hadoop','Database',19989,18451);
INSERT INTO PROJECT VALUES(20,3,'GFS','SuperComputing',19888,27442);
INSERT INTO PROJECT VALUES(23,5,'BSDE','SuperComputing',20012,17911);
INSERT INTO PROJECT VALUES(2,4,'Hive','Database',19974,19970);
INSERT INTO PROJECT VALUES(26,8,'EC3','Cloud',20005,29427);
INSERT INTO PROJECT VALUES(27,5,'Rapyuta','Novel',19886,24622);
INSERT INTO PROJECT VALUES(28,2,'Steam','Novel',20146,16704);
INSERT INTO PROJECT VALUES(29,2,'MS-IIS','Web',20121,29424);
INSERT INTO PROJECT VALUES(5,4,'Hbase','Database',20155,16303);
INSERT INTO PROJECT VALUES(9,5,'Flume','Cloud',19838,27500);
```

addapplication.sql

```
INSERT INTO APPLICATION VALUES(10,'Scarlet',43504,20100815,5);
INSERT INTO APPLICATION VALUES(12,'Jada',45109,20100815,26);
INSERT INTO APPLICATION VALUES(19,'Barbara',42196,20131027,12);
INSERT INTO APPLICATION VALUES(1,'Latifah',17924,20100815,2);
INSERT INTO APPLICATION VALUES(20,'Naomi',39388,20100815,20);
INSERT INTO APPLICATION VALUES(22,'Amena',13920,20131027,28);
INSERT INTO APPLICATION VALUES(23,'Tanya',26101,20131027,1);
INSERT INTO APPLICATION VALUES(29,'Cheyenne',47991,20100815,11);
INSERT INTO APPLICATION VALUES(30,'Amethyst',40597,20131027,12);
INSERT INTO APPLICATION VALUES(34,'Shaine',22812,20070603,23);
INSERT INTO APPLICATION VALUES(36,'April',13297,20100815,13);
INSERT INTO APPLICATION VALUES(38,'Dara',38553,20070603,2);
INSERT INTO APPLICATION VALUES(41,'Destiny',23278,20100815,27);
INSERT INTO APPLICATION VALUES(42,'Melodie',49075,20100815,12);
INSERT INTO APPLICATION VALUES(43,'Amaya',23482,20100815,15);
INSERT INTO APPLICATION VALUES(45,'Zia',43573,20100815,27);
INSERT INTO APPLICATION VALUES(55,'Jolene',20211,20070603,5);
INSERT INTO APPLICATION VALUES(56,'Katelyn',13486,20131027,29);
INSERT INTO APPLICATION VALUES(59,'Briar',19936,20100815,20);
INSERT INTO APPLICATION VALUES(60,'Leopard',49020,20131028,26);
INSERT INTO APPLICATION VALUES(61,'Mavericks',47820,20131029,27);
INSERT INTO APPLICATION VALUES(62,'Tiger',20880,20131030,28);
INSERT INTO APPLICATION VALUES(63,'Lion',24900,20131031,29);
```

addengineer.sql

```
INSERT INTO ENGINEER VALUES(102,'Chanshu Kun',40600,5);
INSERT INTO ENGINEER VALUES(103,'Woods Shao',18000,3);
INSERT INTO ENGINEER VALUES(104,'Lai Yiping',33850,1);
INSERT INTO ENGINEER VALUES(107,'Chang wei Ling',27850,6);
INSERT INTO ENGINEER VALUES(10,'Chan Ling Yun',28350,1);
INSERT INTO ENGINEER VALUES(110,'Wujun Xian',26100,2);
INSERT INTO ENGINEER VALUES(112,'Chan Chi',18600,1);
INSERT INTO ENGINEER VALUES(115,'Liu Yahui',19600,7);
INSERT INTO ENGINEER VALUES(117,'Chan Pei Heng',22900,1);
INSERT INTO ENGINEER VALUES(119,'Hu Ting Xu',26550,3);
INSERT INTO ENGINEER VALUES(120,'Zheng Yueci',40300,7);
INSERT INTO ENGINEER VALUES(122,'Jane Xiu',46100,4);
INSERT INTO ENGINEER VALUES(123,'Qiu Xingyin',20200,7);
INSERT INTO ENGINEER VALUES(124,'Song Yi Hai',48900,1);
INSERT INTO ENGINEER VALUES(126,'Li Zhaolun',43800,2);
INSERT INTO ENGINEER VALUES(127,'Tsai Chi Chang',16200,1);
INSERT INTO ENGINEER VALUES(128,'Wang Ting-jun',28300,8);
INSERT INTO ENGINEER VALUES(129,'Huang Xinyi',34200,5);
INSERT INTO ENGINEER VALUES(130,'Li Weiliang',12600,3);
INSERT INTO ENGINEER VALUES(131,'Linyi Xiong',16900,5);
INSERT INTO ENGINEER VALUES(133,'Zeng Shi Hui',10000,7);
INSERT INTO ENGINEER VALUES(138,'Zhanshao Fu',28150,8);
INSERT INTO ENGINEER VALUES(140,'Zhang Ruyi',30500,6);
INSERT INTO ENGINEER VALUES(142,'Huang Qixin',45650,8);
INSERT INTO ENGINEER VALUES(145,'Liu Zhenjing',31400,8);
INSERT INTO ENGINEER VALUES(148,'Maguang Yang',41700,8);
INSERT INTO ENGINEER VALUES(150,'Liang Weizhe',47250,8);
INSERT INTO ENGINEER VALUES(16,'Wu Jiarong',44050,3);
INSERT INTO ENGINEER VALUES(17,'Ye Huanbang',35850,2);
INSERT INTO ENGINEER VALUES(19,'Dai Hui as',12200,2);
INSERT INTO ENGINEER VALUES(21,'Liupei Xuan',26300,5);
INSERT INTO ENGINEER VALUES(22,'Jiang Jiaxiang',41400,7);
INSERT INTO ENGINEER VALUES(23,'Yao Jing can',36250,5);
INSERT INTO ENGINEER VALUES(27,'Wengqing Rong',30800,1);
INSERT INTO ENGINEER VALUES(28,'The mining hui',15000,3);
INSERT INTO ENGINEER VALUES(30,'Xuya Ting',31250,8);
INSERT INTO ENGINEER VALUES(32,'Hung Chih-shan',32450,8);
INSERT INTO ENGINEER VALUES(33,'Cai Junting',26000,2);
INSERT INTO ENGINEER VALUES(34,'Zhang Yi Xiu',35650,5);
INSERT INTO ENGINEER VALUES(35,'Sun Zhijie',49350,5);
INSERT INTO ENGINEER VALUES(37,'Yang Shuting',18200,1);
INSERT INTO ENGINEER VALUES(38,'Wu Changyan',39800,7);
INSERT INTO ENGINEER VALUES(39,'Limei Chan',38200,4);
INSERT INTO ENGINEER VALUES(40,'Zhang Xin',27400,1);
INSERT INTO ENGINEER VALUES(41,'Cai Rujing',42900,2);
INSERT INTO ENGINEER VALUES(42,'Huang Xinyi',30200,4);
INSERT INTO ENGINEER VALUES(44,'Hui Zhen',39350,3);
INSERT INTO ENGINEER VALUES(46,'Wen-Haur Shieh',27500,5);
INSERT INTO ENGINEER VALUES(48,'Lin Yin',11000,6);
INSERT INTO ENGINEER VALUES(49,'Shishu Fang',18600,3);
INSERT INTO ENGINEER VALUES(4,'Chan Yankang',31450,1);
INSERT INTO ENGINEER VALUES(50,'CaiChang Xing',15700,4);
INSERT INTO ENGINEER VALUES(52,'Ji Mei Xiong',45900,1);
INSERT INTO ENGINEER VALUES(53,'Lo Shu',29000,5);
INSERT INTO ENGINEER VALUES(54,'Chanyu Kang',30850,1);
INSERT INTO ENGINEER VALUES(56,'Qian Ying Xiu',19100,4);
INSERT INTO ENGINEER VALUES(57,'Zhuo Yan Yun',40400,2);
INSERT INTO ENGINEER VALUES(58,'Tong Huimin',33050,2);
INSERT INTO ENGINEER VALUES(5,'Caizi Jun',14600,6);
INSERT INTO ENGINEER VALUES(60,'Yang Yanling',45700,7);
INSERT INTO ENGINEER VALUES(61,'Chang Pei as',24100,2);
INSERT INTO ENGINEER VALUES(62,'Chan Zhewei',30000,5);
INSERT INTO ENGINEER VALUES(63,'Hong Lingkai',47900,4);
INSERT INTO ENGINEER VALUES(64,'Lin Wanjun',24200,5);
INSERT INTO ENGINEER VALUES(69,'Wang Lake',41650,5);
INSERT INTO ENGINEER VALUES(6,'Shu Lake',22500,6);
INSERT INTO ENGINEER VALUES(70,'Lin Yuting',45750,2);
INSERT INTO ENGINEER VALUES(74,'Cai Yiting',37500,4);
INSERT INTO ENGINEER VALUES(75,'Liu Shuhui',29900,2);
INSERT INTO ENGINEER VALUES(76,'Wang Wenxuan',37750,1);
INSERT INTO ENGINEER VALUES(78,'Yang Dengcan',45800,2);
INSERT INTO ENGINEER VALUES(79,'Wang Shujuan',32800,7);
INSERT INTO ENGINEER VALUES(7,'Yangzi Hong',12100,8);
```

```
INSERT INTO ENGINEER VALUES(80,'Chan Xuan Yu',13100,1);
INSERT INTO ENGINEER VALUES(81,'Shimei Jun',34950,3);
INSERT INTO ENGINEER VALUES(85,'Lee Yoo Chun',32800,7);
INSERT INTO ENGINEER VALUES(86,'Jiang Zhongru',44050,3);
INSERT INTO ENGINEER VALUES(88,'Huang Hengrong',19900,2);
INSERT INTO ENGINEER VALUES(89,'Yip Guizhou',30300,6);
INSERT INTO ENGINEER VALUES(8,'Yuan Zi Xiu',44650,7);
INSERT INTO ENGINEER VALUES(90,'Lin Jiahong',12000,8);
INSERT INTO ENGINEER VALUES(91,'Han Jiaxin',16900,6);
INSERT INTO ENGINEER VALUES(92,'Yangming Hong',40850,1);
INSERT INTO ENGINEER VALUES(93,'Wang Yaling',49700,2);
INSERT INTO ENGINEER VALUES(95,'Dingwei Chun',35000,1);
INSERT INTO ENGINEER VALUES(96,'Zhangjia Jiang',28250,6);
INSERT INTO ENGINEER VALUES(98,'Laishan En',26800,1);
INSERT INTO ENGINEER VALUES(99,'Hong Meihui',31050,2);
INSERT INTO ENGINEER VALUES(9,'Li Weiting',14800,5);
```

addmaintain.sql

```
INSERT INTO MAINTAIN VALUES(19,7);
INSERT INTO MAINTAIN VALUES(19,93);
INSERT INTO MAINTAIN VALUES(20,150);
INSERT INTO MAINTAIN VALUES(30,10);
INSERT INTO MAINTAIN VALUES(34,22);
INSERT INTO MAINTAIN VALUES(34,7);
INSERT INTO MAINTAIN VALUES(41,124);
INSERT INTO MAINTAIN VALUES(41,91);
INSERT INTO MAINTAIN VALUES(43,19);
INSERT INTO MAINTAIN VALUES(43,35);
INSERT INTO MAINTAIN VALUES(45,60);
INSERT INTO MAINTAIN VALUES(20,122);
INSERT INTO MAINTAIN VALUES(23,122);
INSERT INTO MAINTAIN VALUES(29,122);
INSERT INTO MAINTAIN VALUES(30,122);
INSERT INTO MAINTAIN VALUES(34,122);
INSERT INTO MAINTAIN VALUES(36,122);
INSERT INTO MAINTAIN VALUES(10,17);
INSERT INTO MAINTAIN VALUES(12,17);
INSERT INTO MAINTAIN VALUES(19,17);
INSERT INTO MAINTAIN VALUES(1,17);
INSERT INTO MAINTAIN VALUES(59,10);
INSERT INTO MAINTAIN VALUES(60,10);
INSERT INTO MAINTAIN VALUES(61,10);
INSERT INTO MAINTAIN VALUES(62,10);
INSERT INTO MAINTAIN VALUES(63,10);
```

addresearch.sql

```
INSERT INTO RESEARCH VALUES(11,110);
INSERT INTO RESEARCH VALUES(11,130);
INSERT INTO RESEARCH VALUES(11,89);
INSERT INTO RESEARCH VALUES(11,91);
INSERT INTO RESEARCH VALUES(11,21);
INSERT INTO RESEARCH VALUES(12,110);
INSERT INTO RESEARCH VALUES(12,130);
INSERT INTO RESEARCH VALUES(12,150);
INSERT INTO RESEARCH VALUES(12,39);
INSERT INTO RESEARCH VALUES(12,49);
INSERT INTO RESEARCH VALUES(12,58);
INSERT INTO RESEARCH VALUES(13,81);
INSERT INTO RESEARCH VALUES(13,37);
INSERT INTO RESEARCH VALUES(13,46);
INSERT INTO RESEARCH VALUES(13,138);
INSERT INTO RESEARCH VALUES(13,150);
INSERT INTO RESEARCH VALUES(13,56);
INSERT INTO RESEARCH VALUES(13,28);
INSERT INTO RESEARCH VALUES(13,99);
INSERT INTO RESEARCH VALUES(15,119);
INSERT INTO RESEARCH VALUES(15,124);
INSERT INTO RESEARCH VALUES(15,150);
INSERT INTO RESEARCH VALUES(1,69);
INSERT INTO RESEARCH VALUES(1,16);
INSERT INTO RESEARCH VALUES(1,23);
INSERT INTO RESEARCH VALUES(1,140);
INSERT INTO RESEARCH VALUES(1,28);
INSERT INTO RESEARCH VALUES(1,50);
INSERT INTO RESEARCH VALUES(1,7);
INSERT INTO RESEARCH VALUES(1,10);
INSERT INTO RESEARCH VALUES(20,138);
INSERT INTO RESEARCH VALUES(20,150);
INSERT INTO RESEARCH VALUES(20,129);
INSERT INTO RESEARCH VALUES(20,49);
INSERT INTO RESEARCH VALUES(20,4);
INSERT INTO RESEARCH VALUES(20,61);
INSERT INTO RESEARCH VALUES(23,80);
INSERT INTO RESEARCH VALUES(23,95);
INSERT INTO RESEARCH VALUES(23,142);
INSERT INTO RESEARCH VALUES(23,133);
INSERT INTO RESEARCH VALUES(2,80);
INSERT INTO RESEARCH VALUES(2,95);
INSERT INTO RESEARCH VALUES(2,142);
INSERT INTO RESEARCH VALUES(2,133);
INSERT INTO RESEARCH VALUES(2,69);
INSERT INTO RESEARCH VALUES(26,46);
INSERT INTO RESEARCH VALUES(26,80);
INSERT INTO RESEARCH VALUES(26,50);
INSERT INTO RESEARCH VALUES(26,41);
INSERT INTO RESEARCH VALUES(26,148);
INSERT INTO RESEARCH VALUES(26,35);
INSERT INTO RESEARCH VALUES(27,102);
INSERT INTO RESEARCH VALUES(27,129);
INSERT INTO RESEARCH VALUES(28,115);
INSERT INTO RESEARCH VALUES(28,131);
INSERT INTO RESEARCH VALUES(28,145);
INSERT INTO RESEARCH VALUES(28,129);
INSERT INTO RESEARCH VALUES(29,61);
INSERT INTO RESEARCH VALUES(29,38);
INSERT INTO RESEARCH VALUES(29,127);
INSERT INTO RESEARCH VALUES(29,130);
INSERT INTO RESEARCH VALUES(29,110);
INSERT INTO RESEARCH VALUES(5,107);
INSERT INTO RESEARCH VALUES(5,69);
INSERT INTO RESEARCH VALUES(5,80);
INSERT INTO RESEARCH VALUES(9,133);
INSERT INTO RESEARCH VALUES(9,145);
INSERT INTO RESEARCH VALUES(9,123);
INSERT INTO RESEARCH VALUES(9,127);
INSERT INTO RESEARCH VALUES(9,138);
INSERT INTO RESEARCH VALUES(9,142);
```